

UNITAT 9 · SETMANA 15

SO natiu per a la IA: agents i LLM al sistema operatiu

 Setmana 15 · dates per confirmar
  Laboratori TD8 (resolt + repte IA)
  4 animacions

LABORATORI TD8 · RESULT (NOU)

Un SO agèntic amb el teu propi micro-LLM

Windows i les distribucions de Linux comencen a acollir agents d'IA dins del sistema. Aquest laboratori construeix aquesta arquitectura en miniatura amb tot el curs: un **micro-LLM** entrenat per al curs converteix peticions en castellà o en anglès (només entén aquestes dues llengües) en una crida a una eina; un **dimoni** (`agentd`) la rep per un sòcol UNIX, la valida amb una **política**, demana **consentiment** si és destructiva i l'executa en un procés tancat per **Landlock**, i ho anota tot en un **registre d'auditoria**.

Compilar i provar

Bash

```
make -C tds                                # també compila td8: tinyllm, agentd, agent,
agent_exec
tds/tests/run.sh td8                        # 24 comprovacions: model, peticions,
política, consentiment, Landlock, auditoria
cd tds/bin/td8 && ./tinyllm -v "list the files in docs" # peticions només
en castellà o en anglès
```

Cal Linux ≥ 5.13 amb Landlock actiu (`cat /sys/kernel/security/lsm`). L'entrenament és opcional (els pesos ja hi són): `make -C tds/td8 train` (NumPy, ≈ 1 h en una CPU).

Informe del laboratori

Entrega aquest laboratori amb un informe curt en LaTeX, amb la plantilla del curs: cada exercici explicat, el codi que importa i el que has executat al terminal. Pots fer servir IA generativa per escriure el LaTeX: declara-ho i verifica cada resultat. Plantilla, exemple i guia de Prism →

EXERCICIS

- | | | | |
|---|--|---|----------------------------------|
| 1 | El micro-LLM: dades i entrenament | 4 | La política i el consentiment |
| 2 | Inferència en C amb <code>mmap</code> i una memòria cau KV | 5 | Defensa en profunditat: Landlock |
| 3 | <code>agentd</code> : sòcol, grup de fils i identitat | 6 | Auditoria i reflexió |

TD8 · 1

● Result · provat

El micro-LLM: dades i entrenament

Enunciat. Llegeix `train/dataset.py` i `train/train.py`. Què aprèn exactament el model? Per què els noms de fitxer es generen a l'atzar? Com sabem que el gradient escrit a mà és correcte?

Solució

Python `tds/td8/train/dataset.py`

```
"""Synthetic data for the TD8 micro-LLM: requests in Spanish or English →
tool call.
```

```
Every example is a text of UTF-8 bytes:
```

```
Q: lista los archivos de docs\n
A: list_files("docs")\n
```

```
The model only learns to translate the intent into ONE call from a closed
catalogue of tools
(the equivalent of the MCP "connectors" of an agentic OS). File and folder
names are generated
at random so that the model learns to COPY them from the request (attention),
not to memorise them.
None of this is safe by itself: the OS decides whether to run the call
(policy + Landlock in agentd).
"""
```

- Un transformer d'estil GPT **a nivell de byte** (vocabulari 256, context 128, $d = 64$, 4 caps, 3 capes, ≈ 175 mil paràmetres), entrenat només amb NumPy (sense PyTorch) i retropropagació escrita a mà; `python3 train.py --check` compara el gradient analític amb diferències finites.

- Aprèn a traduir una petició a **una** crida d'un catàleg tancat de 12 eines. Els noms de fitxer aleatoris obliguen el model a **copiar-los** de la petició (atenció) en lloc de memoritzar-los.
- **Resultat real** (`tds/td8/weights/train_log.json`): 174.656 paràmetres, 12.000 passos (≈ 87 min en 4 fils de CPU), **99,0 %** de coincidència exacta en 500 peticions noves; l'eina més difícil és `write_note` (89 %), perquè s'han de copiar dos arguments. Pesos: 698.688 bytes.
- **Lliçó de dades:** la versió anterior (v1) treia un 99,4 % a la seva prova, però al laboratori va convertir `viejo.txt` en `viefe.txt` i no va reconèixer `atajo`: havia vist pocs noms com aquests. En lloc de canviar la xarxa, es va ampliar el generador de dades (noms amb tot l'alfabet, sense extensió). El model només és tan bo com les seves dades, i per això `agentd` no s'hi fia mai.
- Les dades inclouen a propòsit camins perillosos (`/etc/passwd` , `../..`): el model els ha de copiar fidelment, perquè decidir si són acceptables **no és feina seva**.

Mira-ho per dins a l'animació A9.2 →

TD8 · 2

● Result · provat

Inferència en C amb `mmap` i una memòria cau KV

Enunciat. Explica com carrega `tinylm.c` els pesos sense copiar-los i per què cada token nou costa una sola passada per la xarxa. Executa `./tinylm -v` i relaciona'n les mètriques amb les unitats 4 i 6. Comprova que C i NumPy calculen el mateix.

Solució

C `tds/td8/tinylm.c`

```
int tl_load(tl_model *m, const char *path)
{
    memset(m, 0, sizeof *m);
    int fd = open(path, O_RDONLY);
    if (fd == -1)
        return -1;
    struct stat st;
    if (fstat(fd, &st) == -1 || st.st_size < HEADER) {
        close(fd);
        errno = EINVAL;
    }
```

```

        return -1;
    }

    /* MAP_PRIVATE + PROT_READ: nobody can modify the weights through this
    mapping. */
    void *map = mmap(NULL, (size_t)st.st_size, PROT_READ, MAP_PRIVATE, fd,
0);
    close(fd);                                /* the mapping keeps the
file alive */
    if (map == MAP_FAILED)
        return -1;

    const unsigned char *raw = map;
    int32_t h[7];
    memcpy(h, raw + 8, sizeof h);
    if (memcmp(raw, "FSOLLM01", 8) != 0 || h[0] != 1 || h[4] > TL_MAX_LAYERS
|| h[3] % h[5] != 0) {
        munmap(map, (size_t)st.st_size);
        errno = EINVAL;
        return -1;
    }
    m->vocab = h[1]; m->ctx = h[2]; m->d = h[3]; m->n_layer = h[4]; m->n_head
= h[5]; m->d_ff = h[6];
    m->map = map;
    m->map_size = (size_t)st.st_size;

    /* The pointers point INTO the mapping: no copy (the file is little-
endian float32). */
    const float *p = (const float *)(raw + HEADER);
    const int d = m->d, f = m->d_ff;
#define TAKE(dst, n) do { (dst) = p; p += (n); } while (0)
    TAKE(m->wte, (long)m->vocab * d);
    TAKE(m->wpe, (long)m->ctx * d);
    for (int l = 0; l < m->n_layer; l++) {
        tl_layer *L = &m->layer[l];
        TAKE(L->ln1_g, d); TAKE(L->ln1_b, d);
        TAKE(L->w_qkv, (long)d * 3 * d); TAKE(L->b_qkv, 3 * d);
        TAKE(L->w_o, (long)d * d); TAKE(L->b_o, d);
        TAKE(L->ln2_g, d); TAKE(L->ln2_b, d);
        TAKE(L->w_fc, (long)d * f); TAKE(L->b_fc, f);
        TAKE(L->w_pr, (long)f * d); TAKE(L->b_pr, d);
    }
    TAKE(m->lnf_g, d); TAKE(m->lnf_b, d);
#undef TAKE
    m->n_params = (long)(p - (const float *)(raw + HEADER));
    if ((size_t)HEADER + (size_t)m->n_params * sizeof(float) != m->map_size)

```

```

{
    munmap(map, m→map_size);
    errno = EINVAL;
    return -1;
}
return 0;
}

```

Sortida real

```

$ ./tinylm -v "lista los archivos de docs"
step 0: 'l' 100.0% 'f' 0.0% 'r' 0.0%
step 1: 'i' 100.0% 'q' 0.0% '4' 0.0%
step 2: 's' 100.0% 'g' 0.0% 'q' 0.0%
step 3: 't' 100.0% 'k' 0.0% 'b' 0.0%
step 4: '_' 100.0% '-' 0.0% 'k' 0.0%
step 5: 'f' 100.0% '2' 0.0% 'c' 0.0%
step 6: 'i' 100.0% 'o' 0.0% 'n' 0.0%
step 7: 'l' 100.0% 'L' 0.0% 's' 0.0%
step 8: 'e' 100.0% 'o' 0.0% 'f' 0.0%
step 9: 's' 100.0% '(' 0.0% 'q' 0.0%
step 10: '(' 100.0% ' ' 0.0% 'g' 0.0%
step 11: '"' 100.0% ' ' 0.0% 'R' 0.0%
step 12: 'd' 100.0% 'k' 0.0% 'b' 0.0%
step 13: 'o' 100.0% 'i' 0.0% 'e' 0.0%
step 14: 'c' 100.0% 'k' 0.0% 'f' 0.0%
step 15: 's' 100.0% 'q' 0.0% 'f' 0.0%
step 16: '"' 100.0% 'z' 0.0% 'k' 0.0%
step 17: ')' 100.0% ',' 0.0% ':' 0.0%
step 18: '\n' 100.0% 'A' 0.0% ' ' 0.0%
list_files("docs")

```

```

model      : 174656 parameters, 3 layers, 4 heads, d=64, context 128 bytes
weights    : 698688 bytes mapped with mmap (MAP_PRIVATE, PROT_READ)
KV cache   : 201984 bytes per request (+ scratch)
tokens     : 33 of prompt (prefill) + 19 generated (decode)
time       : load 0.042 ms, inference 9.08 ms (5730 tokens/s)
page faults: 2 during load, 47 during the first inference (minor; major: 0)

```

Sortida real

```

$ python3 train/compare.py ./tinylm 200
200/200 respuestas idénticas entre NumPy y C

```

`mmap` amb `PROT_READ` i `MAP_PRIVATE` retorna una adreça; els punters de cada capa apunten *dins* del fitxer mapat. No es llegeix res fins que el primer token toca els pesos: per això les fallades de pàgina apareixen durant la primera inferència i no durant la càrrega. La memòria cau KV guarda, per capa i per posició, les claus i els valors ja calculats: cada byte nou només calcula la seva fila. La inferència en C i la de NumPy donen **les mateixes respostes** en 200 peticions noves.

TD8 · 3

● Result · provat

agentd : sòcol, grup de fils i identitat

Enunciat. Engaga `agentd` amb un espai de treball i envia-li peticions amb `agent` (en castellà o en anglès). Identifica al codi quina part ve de cada unitat del curs. Com sap `agentd` qui li parla?

Solució

C tds/td8/agentd.c

```
static void serve(int client)
{
    struct ucred cred = { 0 };
    socklen_t len = sizeof cred;
    if (getsockopt(client, SOL_SOCKET, SO_PEERCRED, &cred, &len) == -1)
        return;

    char req[MAX_REQ], out[256], why[PATH_MAX + 64];
    if (read_line(client, req, sizeof req) == 0)
        return;

    /* 1. The model proposes. */
    int steps = tl_generate(&model, req, out, sizeof out, NULL, NULL);
    if (steps < 0) {
        send_str(client, "error: request too long\n");
        audit(cred.uid, cred.pid, req, "-", "deny", "too long");
        return;
    }
    send_fmt(client, "model: %s\n", out);

    /* 2. The operating system disposes. */
    call_t c;
```

```

    if (parse_call(out, &c) == -1) {
        send_str(client, "policy: deny (the model produced an invalid
call)\n");
        audit(cred.uid, cred.pid, req, out, "deny", "invalid call");
        return;
    }
    decision_t d = policy(&c, why, sizeof why);
    send_fmt(client, "policy: %s (%s)\n", DECISION[d], why);
    if (d == DENY) {
        audit(cred.uid, cred.pid, req, out, "deny", why);
        return;
    }
    if (d == ASK) {
        char answer[16];
        send_fmt(client, "ask: allow the agent to run %s? [y/N]\n", out);
        read_line(client, answer, sizeof answer);
        if (answer[0] != 'y' && answer[0] != 'Y') {
            send_str(client, "cancelled by the user\n");
            audit(cred.uid, cred.pid, req, out, "ask", "refused by the
user");
            return;
        }
    }

    /* 3. Sandboxed execution. */
    const char *outcome = run_tool(client, &c);
    audit(cred.uid, cred.pid, req, out, d == ASK ? "ask:approved" : "allow",
outcome);
}

```

Sortida real

```

$ ./agent "lista los archivos de docs"
model: list_files("docs")
policy: allow (tool in the catalogue, path inside the workspace)
[sandbox: Landlock ABI 8, workspace only]
2 entries in docs
-      0  plan.md
-      0  syllabus.pdf

```

Sortida real

```

$ ./agent "muéstrame notas.txt"
model: read_file("notas.txt")
policy: allow (tool in the catalogue, path inside the workspace)
[sandbox: Landlock ABI 8, workspace only]
Laboratorio TD8
fork, exec, pipe, mmap, sockets

```

Sortida real

```

$ ./agent "busca 'error' en log.txt"
model: search("error", "log.txt")
policy: allow (tool in the catalogue, path inside the workspace)
[sandbox: Landlock ABI 8, workspace only]
log.txt:2: error: disk full
log.txt:4: error: timeout
2 matching line(s) for "error"

```

Sortida real

```

$ ./agent "¿qué versión del núcleo tengo?"
model: system_info()
policy: allow (tool in the catalogue, path inside the workspace)
[sandbox: Landlock ABI 8, workspace only]
Linux 7.0.0-34-generic (x86_64), 28 CPUs
memory: 32.8 GB total, 6.9 GB free; 3110 processes; up 4 h 7 min

```

Sortida real

```

$ for i in $(seq 8); do ./agent "lista mis archivos" & done; wait
8

```

PEÇA D'AGENTD	UNITAT
Sòcol UNIX amb permisos 0600	U8 · U3
<code>SO_PEERCRED</code> : uid i pid del client, que dona el nucli	U8
Grup de fils amb una cua limitada (semàfors + mutex)	U7
Pesos compartits amb <code>mmap</code>	U6
<code>fork</code> + <code>execv</code> de l'executor; sortida per una canonada	U4 · U5
<code>SIGTERM</code> per aturar; <code>SIGPIPE</code> ignorat	U6

TD8 · 4

● Result · provat

La política i el consentiment

Enunciat. Demana a l'agent que esborri un fitxer de l'espai de treball (responent sí i no), que esborri `/etc/passwd`, que llegeixi `../.. /etc/shadow` i que llegeixi `atajo`, un enllaç simbòlic a `/etc/passwd` dins de l'espai de treball. Qui decideix en cada cas: el model o el SO?

Solució



C tds/td8/agentd.c

```
static int inside_workspace(const char *rel, char *why, size_t whysz)
{
    if (rel[0] == '/' || rel[0] == '~') {
        snprintf(why, whysz, "absolute path outside the workspace");
        return 0;
    }
    if (strcmp(rel, "..") == 0 || strncmp(rel, "../", 3) == 0 || strstr(rel,
"/../") || strstr(rel, "/..") ) {
        snprintf(why, whysz, "'..' leaves the workspace");
        return 0;
    }
    /* Resolve symbolic links: the file (or, if it does not exist yet, its
directory). */
    char full[PATH_MAX * 2], real[PATH_MAX];
    snprintf(full, sizeof full, "%s/%s", workspace, rel);
    if (realpath(full, real) == NULL) {
        char *slash = strrchr(full, '/');
        *slash = '\0';
        if (realpath(full, real) == NULL) {
            snprintf(why, whysz, "directory does not exist");
            return 0;
        }
    }
    size_t n = strlen(workspace);
    if (strncmp(real, workspace, n) != 0 || (real[n] != '\0' && real[n] !=
'/')) {
        snprintf(why, whysz, "resolves to %s, outside the workspace", real);
        return 0;
    }
    return 1;
}

static decision_t policy(const call_t *c, char *why, size_t whysz)
{
    const tool_t *t = c->tool;
    if (strcmp(t->name, "none") == 0) {
        snprintf(why, whysz, "no tool for this request");
        return DENY;
    }
    if (policy_on && t->path_arg ≥ 0 && !inside_workspace(c->args[t
->path_arg], why, whysz))
        return DENY;
    if (t->consent) {
```

```

    snprintf(why, whysz, "destructive action: needs the user's consent");
    return ASK;
}
    snprintf(why, whysz, policy_on ? "tool in the catalogue, path inside the
workspace" : "POLICY DISABLED");
    return ALLOW;
}

```

Sortida real

```

$ ./agent -n "borra viejo.txt"
model: delete_file("viejo.txt")
policy: ask (destructive action: needs the user's consent)
ask: allow the agent to run delete_file("viejo.txt")? [y/N]
> n (automatic answer)
cancelled by the user

```

Sortida real

```

$ ./agent -y "borra viejo.txt"
model: delete_file("viejo.txt")
policy: ask (destructive action: needs the user's consent)
ask: allow the agent to run delete_file("viejo.txt")? [y/N]
> y (automatic answer)
[sandbox: Landlock ABI 8, workspace only]
viejo.txt deleted

```

Sortida real

```

$ ./agent "borra /etc/passwd"
model: delete_file("/etc/passwd")
policy: deny (absolute path outside the workspace)

```

Sortida real

```

$ ./agent "lee ../../etc/shadow"
model: read_file("../../etc/shadow")
policy: deny ('..' leaves the workspace)

```

Sortida real

```
$ ./agent "lee atajo"
model: read_file("atajo")
policy: deny (resolves to /etc/passwd, outside the workspace)
```

Sortida real

```
$ ./agent "cuéntame un chiste"
model: none()
policy: deny (no tool for this request)
```

El model sempre proposa el que se li ha demanat —incloent-hi

`delete_file("/etc/passwd")` —; qui decideix és `agentd`: els camins absoluts, `..` i els enllaços que s'escapen (resolts amb `realpath`) es deneguen; tot el que és destructiu ho pregunta a l'usuari.

Prova-ho amb les teves pròpies peticions a l'animació A9.4 (en castellà o en anglès) →

TD8 · 5

● Result · provat

Defensa en profunditat: Landlock

Enunciat. Torna a engegar `agentd` amb `--no-policy` (simulant un error a la política) i repeteix els atacs. Després executa `agent_exec` directament amb `--no-sandbox` i sense. Quina capa t'atura ara?

Solució

C tds/td8/agent_exec.c

```
static int sandbox(const char *workspace)
{
    int abi = ll_create(NULL, 0, LANDLOCK_CREATE_RULESET_VERSION);
    if (abi < 1)
        return 0;

    __u64 fs = LANDLOCK_ACCESS_FS_EXECUTE | LANDLOCK_ACCESS_FS_WRITE_FILE |
        LANDLOCK_ACCESS_FS_READ_FILE |
```

```

        LANDLOCK_ACCESS_FS_READ_DIR | LANDLOCK_ACCESS_FS_REMOVE_DIR |
LANDLOCK_ACCESS_FS_REMOVE_FILE |
        LANDLOCK_ACCESS_FS_MAKE_CHAR | LANDLOCK_ACCESS_FS_MAKE_DIR |
LANDLOCK_ACCESS_FS_MAKE_REG |
        LANDLOCK_ACCESS_FS_MAKE_SOCKET | LANDLOCK_ACCESS_FS_MAKE_FIFO |
LANDLOCK_ACCESS_FS_MAKE_BLOCK |
        LANDLOCK_ACCESS_FS_MAKE_SYM;

    if (abi ≥ 2) fs |= LANDLOCK_ACCESS_FS_REFER;
    if (abi ≥ 3) fs |= LANDLOCK_ACCESS_FS_TRUNCATE;
    if (abi ≥ 5) fs |= LANDLOCK_ACCESS_FS_IOCTL_DEV;

    struct landlock_ruleset_attr attr = { .handled_access_fs = fs };
    size_t size = sizeof(__u64); /* ABI 1-3:
only the fs field */
    if (abi ≥ 4) {
        attr.handled_access_net = LANDLOCK_ACCESS_NET_BIND_TCP |
LANDLOCK_ACCESS_NET_CONNECT_TCP;
        size = offsetof(struct landlock_ruleset_attr, handled_access_net) +
sizeof(__u64);
    }
    if (abi ≥ 6) {
        attr.scoped = LANDLOCK_SCOPE_ABSTRACT_UNIX_SOCKET |
LANDLOCK_SCOPE_SIGNAL;
        size = sizeof attr;
    }
    int rs = ll_create(&attr, size, 0);
    exit_if(rs == -1, "landlock_create_ruleset");

    /* Everything inside the workspace (minus executing programs), read-only
/proc. */
    exit_if(ll_add_path(rs, workspace, fs & ~
(__u64)LANDLOCK_ACCESS_FS_EXECUTE) == -1, "landlock_add_rule workspace");
    exit_if(ll_add_path(rs, "/proc", LANDLOCK_ACCESS_FS_READ_FILE |
LANDLOCK_ACCESS_FS_READ_DIR) == -1,
        "landlock_add_rule /proc");

    exit_if(prctl(PR_SET_NO_NEW_PRIVS, 1, 0, 0, 0) == -1, "prctl");
    exit_if(syscall(SYS_landlock_restrict_self, rs, 0) == -1,
"landlock_restrict_self");
    close(rs);
    return abi;
}

static void limits(void)
{

```

```
struct rlimit cpu = { 2, 3 }, fsize = { 1 << 20, 1 << 20 }, nofile = {
64, 64 };
exit_if(setrlimit(RLIMIT_CPU, &cpu) = -1, "setrlimit cpu");      /*
SIGXCPU after 2 s */
exit_if(setrlimit(RLIMIT_FSIZE, &fsize) = -1, "setrlimit fsize"); /*
SIGXFSZ past 1 MiB */
exit_if(setrlimit(RLIMIT_NOFILE, &nofile) = -1, "setrlimit nofile");
alarm(5);                                                        /*
SIGALRM: real time */
}
```

Sortida real

```
$ ./agent "lee atajo"
model: read_file("atajo")
policy: allow (POLICY DISABLED)
[sandbox: Landlock ABI 8, workspace only]
read_file: atajo: Permission denied
```

Sortida real

```
$ ./agent "lee /etc/passwd"
model: read_file("/etc/passwd")
policy: allow (POLICY DISABLED)
[sandbox: Landlock ABI 8, workspace only]
read_file: /etc/passwd: Permission denied
```

Sortida real

```
$ ./agent_exec workspace read_file /etc/hostname; ./agent_exec --no-sandbox
workspace read_file /etc/hostname
[sandbox: Landlock ABI 8, workspace only]
read_file: /etc/hostname: Permission denied
[sandbox: DISABLED]
host
```

Abans de tocar res, `agent_exec` es tanca: límits de CPU, de mida de fitxer i de descriptors (`setrlimit`), una alarma de 5 s, `no_new_privs` i una regla de Landlock que només permet l'espai de treball (i `/proc` de només lectura), denega les connexions TCP i els senyals fora de l'entorn aïllat. Fins i tot amb la política desactivada, el **nucli** respon `Permission denied`. El mateix programa, sense entorn aïllat, sí que llegeix el fitxer.

TD8 · 6

● Result · provat

Auditoria i reflexió

Enunciat. Llegeix `audit.log`. Quina informació permet reconstruir què ha fet l'agent i en nom de qui? Què li falta a aquest SO agèntic per fer-lo servir de debò?

Solució



Sortida real

```

2026-09-30T13:14:12 uid=1000 pid=364726 request="lista los archivos de docs"
call=list_files("docs") decision=allow outcome=ok
2026-09-30T13:14:13 uid=1000 pid=364735 request="muéstrame notas.txt"
call=read_file("notas.txt") decision=allow outcome=ok
2026-09-30T13:14:13 uid=1000 pid=364744 request="¿cuántas líneas tiene
main.c?" call=count_lines("main.c") decision=allow outcome=ok
2026-09-30T13:14:13 uid=1000 pid=364753 request="busca 'error' en log.txt"
call=search("error", "log.txt") decision=allow outcome=ok
2026-09-30T13:14:13 uid=1000 pid=364762 request="anota 'examen el lunes' en
agenda.md" call=write_note("agenda.md", "examen el lunes") decision=allow
outcome=ok
2026-09-30T13:14:13 uid=1000 pid=364771 request="crea la carpeta proyectos"
call=make_dir("proyectos") decision=allow outcome=ok
2026-09-30T13:14:13 uid=1000 pid=364780 request="¿cuánto espacio libre
queda?" call=disk_usage() decision=allow outcome=ok
2026-09-30T13:14:13 uid=1000 pid=364789 request="¿qué versión del núcleo
tengo?" call=system_info() decision=allow outcome=ok
2026-09-30T13:14:13 uid=1000 pid=364798 request="show running processes"
call=processes() decision=allow outcome=ok
2026-09-30T13:14:13 uid=1000 pid=364807 request="borra viejo.txt"
call=delete_file("viejo.txt") decision=ask outcome=refused by the user
2026-09-30T13:14:13 uid=1000 pid=364815 request="borra viejo.txt"
call=delete_file("viejo.txt") decision=ask:approved outcome=ok
2026-09-30T13:14:13 uid=1000 pid=364824 request="borra /etc/passwd"
call=delete_file("/etc/passwd") decision=deny outcome=absolute path outside
the workspace
2026-09-30T13:14:13 uid=1000 pid=364832 request="lee ../../etc/shadow"
call=read_file("../../etc/shadow") decision=deny outcome='..' leaves the
workspace
2026-09-30T13:14:13 uid=1000 pid=364840 request="lee atajo"
call=read_file("atajo") decision=deny outcome=resolves to /etc/passwd,
outside the workspace
2026-09-30T13:14:13 uid=1000 pid=364848 request="cuéntame un chiste"
call=none() decision=deny outcome=no tool for this request
2026-09-30T13:14:13 uid=1000 pid=364873 request="lista mis archivos"
call=list_files(".") decision=allow outcome=ok
2026-09-30T13:14:13 uid=1000 pid=364875 request="lista mis archivos"
call=list_files(".") decision=allow outcome=ok
2026-09-30T13:14:13 uid=1000 pid=364877 request="lista mis archivos"
call=list_files(".") decision=allow outcome=ok
2026-09-30T13:14:13 uid=1000 pid=364876 request="lista mis archivos"
call=list_files(".") decision=allow outcome=ok
2026-09-30T13:14:13 uid=1000 pid=364880 request="lista mis archivos"

```

```

call=list_files(".") decision=allow outcome=ok
2026-09-30T13:14:13 uid=1000 pid=364879 request="lista mis archivos"
call=list_files(".") decision=allow outcome=ok
2026-09-30T13:14:13 uid=1000 pid=364881 request="lista mis archivos"
call=list_files(".") decision=allow outcome=ok
2026-09-30T13:14:13 uid=1000 pid=364878 request="lista mis archivos"
call=list_files(".") decision=allow outcome=ok

```

- Cada línia té la data, l'uid i el pid del client (que dona el nucli, no el client), la petició, la crida proposada, la decisió i el resultat: distingeix què ha fet l'agent del que ha demanat l'usuari, com exigeix Windows amb els seus comptes d'agent.
- Li falten, entre altres coses: un usuari propi per a l'agent (avui s'executa com el docent), un filtre `seccomp`, límits de memòria amb cgroups, protecció del registre davant del mateix agent i una defensa contra la injecció de prompt quan l'agent llegeixi el contingut dels fitxers.

Llista de comprovació

Repte amb IA generativa · Una eina nova per a l'agent... i un atac



Objectiu. Afegeix una eina `copy_file(source, destination)` al SO agèntic: genera dades, torna a entrenar el micro-LLM, amplia la política i l'executor. Després intenta que l'agent copii un fitxer fora de l'espai de treball i comprova quina capa l'atura.

Prompt inicial. Fes-lo servir a l'eina que prefereixis (Claude, ChatGPT, Gemini, Copilot, Codex...) amb la versió *resolta* del laboratori com a context. Demana primer un pla i revisa cada pas abans d'acceptar-lo.

Prompt

Treballa amb un SO agèntic de joguina en C: un micro-LLM (`tinyllm.c`) converteix peticions en crides a eines, `agentd` aplica una política i `agent_exec` executa l'eina dins d'un entorn aïllat de Landlock [enganxa `dataset.py`, la taula `TTOOLS` d'`agentd.c` i el `main()` d'`agent_exec.c`]. Vull afegir `copy_file(source, destination)`. Guia'm per: (1) afegir plantilles en castellà i en anglès a `dataset.py` (el micro-LLM només entén aquestes dues llengües); (2) tornar a entrenar i mesurar la precisió abans i després; (3) ampliar la política perquè TOTS DOS arguments quedin dins de l'espai de treball; (4) implementar la còpia a `agent_exec`. Després ajuda'm a dissenyar 5 peticions malicioses (camins absoluts, '..', enllaços simbòlics, injecció al text) i a predir quina capa atura cadascuna: l'analitzador, la política o Landlock.

Com comprovar el resultat

- La precisió del model reentrenat amb les eines anteriors no baixa més de 2 punts (`train.py --eval`).
- `copy_file` amb un destí fora de l'espai de treball acaba en `policy: deny`, i amb `--no-policy` en `Permission denied` (Landlock).
- El fitxer `audit.log` registra els 5 intents amb la seva decisió.

Declara l'ús de la IA: eina, prompts rellevants, què has comprovat tu i què has corregit. El que compta és que puguis explicar i verificar cada decisió.